# Applied Databases

**Final Project - Innovation Document**

*Higher Diploma in Science in Data Analytics*

ATU, Galway – Mayo

---

## 1. Overview

As part of my Applied Databases final project, I developed two innovation features for a Conference Management System built using Python, MySQL, and Neo4j. These features were designed to extend the application beyond the core project requirements while also demonstrating my understanding of database technologies, data integration, and software design principles.

The two innovation features I implemented were:

- **Colour-Coded Terminal Output** - developed using the Python *colorama* library to improve readability and user interaction within the terminal interface.
- **Attendee Recommendation Engine** - a feature that recommends attendees to connect with based on shared session attendance, using data integrated from both MySQL and Neo4j.

---

## 2. Required Packages and Installation

To support the innovation features, I used several additional Python packages within the project. In line with the project specification, these packages are installed automatically when `main.py` is executed. If the automatic installation process does not work, the required packages can also be installed manually using the following command:

```
pip install -r requirements.txt
```

I implemented the automatic installation process at the beginning of `main.py` using Python's subprocess module. This allows the program to call `pip` programmatically and install any missing dependencies before the application launches. By doing this, I ensured that the system can run correctly on any machine, including the Virtual Machine used for assessment, without requiring additional manual configuration from the user.

---

## 3. Innovation Feature 1 - Colour-Coded Terminal Output

### 3.1 Description

The first innovation feature I implemented was colour-coded terminal output throughout the application using the Python *colorama* library **[1][2]**. Instead of displaying all information in plain white text, I assigned distinct and consistent colours to different categories of output.

### 3.2 Motivation and Rationale

In command-line applications, it is important that users can quickly recognise different types of information, such as errors, confirmations, prompts, and general data output. Without colour differentiation, users are required to read each line carefully to understand its purpose. By introducing consistent colour-coding across the application, I enabled users to interpret information quicker and intuitively, reducing cognitive load and improving the overall user experience.

### 3.3 Implementation

To implement this feature, I created a dedicated module called `colours.py` to manage all colour-related functionality. I chose this approach to centralise the use of the *colorama* library within a single file, allowing the rest of the application to simply import reusable helper functions from `colours.py`.

This design decision improved the maintainability of the project because any future changes to the colour scheme can be made in one location without modifying multiple files throughout the application.

The colour scheme used throughout the application is shown below:

| Colour | Helper Function | Applied To |
|---|---|---|
| Cyan | print_header() | All headings and section titles |
| White | print_menu_item() | All menu options |
| Magenta | print_prompt() | All user input prompts |
| Green | print_success() | Success confirmation messages |
| Red | print_error() | All error messages |
| Yellow | print_data_row() | All data output rows |
| Grey / Dim | print_info() | General informational messages |

### 3.4 References

**[1]** colorama - PyPI Package Page

**[2]** colorama - GitHub Repository

---

## 4. Innovation Feature 2 - Attendee Recommendation Engine

### 4.1 Description

The second innovation feature I developed was an **Attendee Recommendation Engine**, which is accessible through Option 7 on the main menu. This feature suggests conference attendees that a user may wish to connect with based on the sessions they have both attended.

The concept is like the *People You May Know* feature used on professional networking platforms such as LinkedIn**,** where shared professional interests and interactions are used to recommend potential new connections.

### 4.2 Motivation and Rationale

I selected the recommendation engine as an innovation feature for several reasons. Firstly, it adds genuine real-world value to the Conference Management System, as identifying professional connections based on shared interests is a common and useful feature in modern networking platforms.

Secondly, this feature allowed me to demonstrate one of the core principles of the Applied Databases module: different database technologies are designed for different purposes, and more powerful applications can be created by combining multiple database systems together effectively.

This feature specifically required the combined use of both MySQL and Neo4j:

- **MySQL** was used to identify attendees who participated in the same conference sessions.
- **Neo4j** was used to identify existing relationships between attendees through graph connections.

Neither database could support the feature fully on its own:

- MySQL alone could identify shared session attendance, but it has no awareness of existing attendee relationships.
- Neo4j alone could identify existing connections, but it does not contain session attendance data.
- By combining both databases, I was able to generate recommendations that were both relevant and non-redundant.

### 4.3 How It Works

The recommendation engine operates through the following sequence of steps:

1. The user enters an Attendee ID
2. MySQL is queried to verify the attendee exists and retrieve their name
3. MySQL is queried via the registration table to retrieve all sessions the attendee has attended
4. MySQL is queried again using a JOIN across the registration, attendee and session tables with an IN clause, to retrieve all other attendees who attended any of those same sessions
5. Neo4j is queried to retrieve all attendees the given attendee is already CONNECTED_TO (in either direction)

6. The already-connected attendees are filtered out from the candidate list, leaving only genuine new recommendations.
7. The resulting recommendations are displayed, showing the attendee ID, name and the session, they have in common.

To improve efficiency and data quality, I used additional Python data structures during processing:

- A Python dictionary was used to remove duplicate recommendations. This was necessary because attendees who shared multiple sessions would otherwise appear multiple times in the MySQL query results. The dictionary ensured that each attendee appeared only once in the final output.
- A Python set was used to store the attendee IDs retrieved from Neo4j. Because sets allow very fast membership checking with O(1) time complexity, this made the filtering process much more efficient, particularly when working with a larger number of attendee connections **[8].**

### 4.4 Database Interaction Summary

| Step | Database | Purpose |
|---|---|---|
| 1-2 | MySQL | Verify attendee exists and retrieve name |
| 3 | MySQL | Get list of sessions attended via registration table |
| 4 | MySQL | Find other attendees at same sessions via JOIN |
| 5 | Neo4j | Get existing CONNECTED_TO relationships |
| 6-7 | Python | Filter and display recommendations |

### 4.5 References

**[3]** MySQL JOIN Documentation

**[4]** MySQL Comparison Operators - IN clause

**[5]** Neo4j Cypher Pattern Reference

**[6]** Neo4j Python Driver Documentation

---

## 5. Full Reference List

**[1]** colorama - PyPI Package Page - https:/pypi.org/project/colorama/

**[2]** colorama - GitHub Repository - https:/github.com/tartley/colorama

**[3]** MySQL JOIN Operator - https://dev.mysql.com/doc/refman/9.7/en/join.html

**[4]** MySQL Comparison Operators - https:/dev.mysql.com/doc/refman/8.0/en/comparison-operators.html

**[5]** Neo4j Cypher Pattern Reference - https:/neo4j.com/docs/cypher-manual/current/patterns/reference/

**[6]** Neo4j Python Driver -https:/neo4j.com/docs/python-manual/current/

**[7]** MySQL SELECT Statement - https:/dev.mysql.com/doc/refman/8.0/en/select.html

**[8]** Python sets - https:/realpython.com/python-sets/